# Isotope tracing for Apache Kafka

Lightweight, end-to-end record tracing for Kafka built entirely on **public extension points** — a `ProducerInterceptor` plus record headers. No broker changes, no agent, no vendor lock-in. Stamp a UUIDv7 trace id and origin metadata on first produce, accumulate a hop on every re-produce, and reconstruct latency / topology / hop-distribution from the headers (or from the optional Prometheus meters).

Two artifacts, so you only take what you need:

| Module | Coordinate | Pulls | Use it for |
|---|---|---|---|
| **isotope-core** | `ai.signalroom:isotope-core` | Jackson, SLF4J (`kafka-clients` is `compileOnly`) | Trace **propagation** — the interceptor, headers, consume markers. |
| **isotope-metrics** | `ai.signalroom:isotope-metrics` | isotope-core + Micrometer/Prometheus | Optional `/metrics` exporter for the stateless reports. |

> `isotope-core` never depends on a metrics library. Emission is routed through a no-op `IsotopeMetricsSink` until `isotope-metrics` registers the Prometheus one — so propagation runs with zero metrics overhead.

## Install (Gradle, GitHub Packages)

```
repositories {
    maven { url 'https://maven.pkg.github.com/j3-signalroom/confluent-kafka-isotope' }
}
dependencies {
    implementation 'ai.signalroom:isotope-core:0.17.1'
    implementation 'ai.signalroom:isotope-metrics:0.17.1' // optional — only for Prometheus
}
```

## Use it

**1. Produce side — register the interceptor.** Every `send()` is stamped/hopped automatically; nothing else to call.

```
props.put(ProducerConfig.INTERCEPTOR_CLASSES_CONFIG,
          IsotopeProducerInterceptor.class.getName());
props.put(IsotopeProducerInterceptor.SERVICE_NAME_CONFIG,  "order-intake-
```

```
service");
props.put(IsotopeProducerInterceptor.PIPELINE_NAME_CONFIG, "orders");
```

**2. Consume side — adopt to continue the trace, or mark a terminal consume.**

```
// A stage that consumes then re-produces ("hop"): adopt so the next send()
// continues the same trace instead of starting a new one.
IsotopeContext.adoptFromRecord(record, "order-enrichment-service");
// ... process and produce downstream; the interceptor reads the adopted
context ...
IsotopeContext.clear(); // per-record, on the same thread

// A terminal consumer (no re-produce): write a bipartite consume-edge
marker.
IsotopeContext.recordConsume(record, "shipping-notification-service",
markerProducer);
```

**3. (Optional) Metrics — start the Prometheus exporter once at boot.**

```
import ai.signalroom.kafka.isotope.metrics.PrometheusIsotopeMetrics;

PrometheusIsotopeMetrics.start(9404); // serves GET /metrics; no-op on a
port clash
```

This serves `isotope_hop_latency_*`, `isotope_hop_records_total`, and the consume-side meters at `http://localhost:9404/metrics`. Until you call `start`, the interceptor's metric calls are inert.

## What's where

- `Isotope` — the header model + JSON codec, UUIDv7, `MAX_HOPS`.
- `IsotopeContext` — thread-local context, `adoptFromRecord`, `recordConsume`.
- `IsotopeProducerInterceptor` — the `ProducerInterceptor` that stamps and hops.
- `IsotopeMetrics` / `IsotopeMetricsSink` — the metrics seam (core stays metrics-free).

For the full meter/PromQL reference and the design rationale (why only three of seven reports are metrics-native), see docs/metrics.md and docs/design.md. The runnable demo and one-command Prometheus + Grafana showcase live in the `app` module and k8s/monitoring.